

Team cO(n)stant — cheat sheet, part 2

Graphs · DP · number theory · strings · Python. All C++ compiled under g++ -std=c++17 -Wall -Wextra and unit-tested (32 assertions).

A Choosing the algorithm · B Union-Find · C Traversal & shortest paths · D MST, SCC, matching, flow · E Standard DP · F Number theory · G Geometry additions · H String matching · I Python · J Before you submit

A Choosing the algorithm

Read the bound → derive the complexity → pick the algorithm

Bound	Target	Technique
$n \leq 10^9$	$O(1), O(\log n), O(\sqrt{n})$	closed form, binary search
$n \leq 10^6$	$O(n), O(n \log n)$	greedy, sort, two pointers, D&C
$n \leq 10^3, W \leq 10^3$	$O(nW)$	DP over an $n \times W$ table
$n \leq 10^2$	$O(n^3)$	Floyd-Warshall, Gauss, interval DP
$n \leq 16$	$O(2^n), O(n \cdot 2^n)$	bitmask brute force / bitmask DP
$n \leq 8$	$O(n!)$	permutations

Budget $\approx 10^7$ steps per test case. Over it → do not start implementing.
Python: allow one complexity class more margin.

Binary search on the answer

If the problem asks for an optimum, ask instead: *can I achieve k?* If check(k) is monotone, binary search it. Turns optimisation into a decision problem.

```
ll lo = 0, hi = HI; // hi must be achievable
while (lo < hi) {
    ll mid = lo + (hi - lo) / 2; // NOT (lo+hi)/2: overflow
    if (check(mid)) hi = mid; else lo = mid + 1;
}
// lo == smallest feasible value
```

Floating point: loop a fixed 100 iterations instead of comparing.

Shortest path: which one?

Situation	Use
unweighted / all weights equal	BFS — $O(V+E)$
non-negative weights	Dijkstra — $O((V+E) \log V)$
negative weights possible	Bellman-Ford — $O(VE)$
all pairs, small n	Floyd-Warshall — $O(V^3)$

B Union-Find (DSU)

Near $O(1)$ amortised. Kruskal, connectivity, cycle detection, grouping.
unite returns false if already joined.

```
struct DSU {
    vector<int> p, sz;
    DSU(int n): p(n), sz(n,1) { iota(p.begin(), p.end(), 0); }
    int find(int x){ while(p[x]!=x){ p[x]=p[p[x]]; x=p[x]; } return x; }
    bool unite(int a,int b){
        a=find(a); b=find(b);
        if(a==b) return false;
        if(sz[a]<sz[b]) swap(a,b);
        p[b]=a; sz[a]+=sz[b]; return true;
    }
};
```

C Traversal & shortest paths

```
vector<int> bfs(vector<vector<int>>& adj, int s){
    vector<int> dist(adj.size(), -1);
    queue<int> q; dist[s]=0; q.push(s);
    while(!q.empty()){
        int v=q.front(); q.pop();
        for(int w: adj[v]) if(dist[w]==-1){
            dist[w]=dist[v]+1; q.push(w); }
    }
    return dist;
}
```

Grid BFS: pad the grid with a wall ring so you never bounds-check. int dx[]={1,-1,0,0}, dy[]={0,0,1,-1};

```
vector<int> dfs_iter(vector<vector<int>>& adj, int s){
    vector<int> seen(adj.size(),0), order;
    vector<int> st{s};
    while(!st.empty()){
        int v=st.back(); st.pop_back();
        if(seen[v]) continue;
        seen[v]=1; order.push_back(v);
        for(int w: adj[v]) if(!seen[w]) st.push_back(w);
    }
    return order;
}
```

Iterative, so no stack overflow on deep graphs.

```
vector<ll> dijkstra(vector<vector<pair<int,int>>& adj, int s){
    const ll INF = LLONG_MAX/4;
    vector<ll> d(adj.size(), INF); d[s]=0;
    priority_queue<pair<ll,int>, vector<pair<ll,int>, greater<>> pq;
    pq.push({0,s});
    while(!pq.empty()){
        auto [dv,v]=pq.top(); pq.pop();
        if(dv>d[v]) continue; // stale entry
        for(auto [w,wt]: adj[v])
            if(dv+wt<d[w]){ d[w]=dv+wt; pq.push({d[w],w}); }
    }
    return d;
}
```

```
bool bellman_ford(int n, vector<array<ll,3>>& edges,
                  int s, vector<ll>& d){
    const ll INF = LLONG_MAX/4;
    d.assign(n, INF); d[s]=0;
    for(int i=0;i<n-1;i++)
        for(auto& e: edges)
            if(d[e[0]]<INF && d[e[0]]+e[2]<d[e[1]])
                d[e[1]]=d[e[0]]+e[2];
    for(auto& e: edges) // one more pass => neg cycle
        if(d[e[0]]<INF && d[e[0]]+e[2]<d[e[1]]) return false;
    return true;
}
```

```
void floyd(vector<vector<ll>>& d){
    int n=d.size();
    for(int k=0;k<n;k++) for(int i=0;i<n;i++) for(int j=0;j<n;j++)
        if(d[i][k]+d[k][j] < d[i][j]) d[i][j]=d[i][k]+d[k][j];
}
```

```
vector<int> toposort(vector<vector<int>>& adj){
    int n=adj.size(); vector<int> indeg(n,0), order;
    for(int v=0;v<n;v++) for(int w: adj[v]) indeg[w]++;
    queue<int> q;
    for(int v=0;v<n;v++) if(!indeg[v]) q.push(v);
    while(!q.empty()){
        int v=q.front(); q.pop(); order.push_back(v);
        for(int w: adj[v]) if(--indeg[w]==0) q.push(w);
    }
    return order; // size < n => graph has a cycle
}
```

D MST, SCC, matching, flow

```
ll kruskal(int n, vector<array<ll,3>> edges){
    sort(edges.begin(), edges.end()); // weight first
    DSU d(n); ll total=0;
    for(auto& e: edges) if(d.unite(e[1],e[2])) total+=e[0];
    return total;
}
```

```
vector<int> scc(vector<vector<int>>& adj){
    int n=adj.size();
    vector<vector<int>> rev(n);
    for(int v=0;v<n;v++) for(int w: adj[v]) rev[w].push_back(v);
    vector<int> seen(n,0), order;
    for(int s=0;s<n;s++){
        if(seen[s]) continue;
        vector<pair<int,int>> st{{s,0}};
        while(!st.empty()){
            auto& [v,i]=st.back();
            if(!i){ if(seen[v]){ st.pop_back(); continue; }
                    seen[v]=1; }
            if(i < (int)adj[v].size()){ int w=adj[v][i++];
                if(!seen[w]) st.push_back({w,0}); }
            else { order.push_back(v); st.pop_back(); }
        }
    }
    vector<int> comp(n,-1); int c=0;
    for(int i=n-1;i>=0;i--){
        int s=order[i];
        if(comp[s]!=-1) continue;
        vector<int> st{s}; comp[s]=c;
        while(!st.empty()){
            int v=st.back(); st.pop_back();
            for(int w: rev[v]) if(comp[w]==-1){
                comp[w]=c; st.push_back(w); }
        }
        c++;
    }
    return comp;
}
```

```
bool try_kuhn(int v, vector<vector<int>>& adj,
             vector<int>& used, vector<int>& mt){
    for(int to: adj[v]){
        if(used[to]) continue;
        used[to]=1;
        if(mt[to]==-1 || try_kuhn(mt[to],adj,used,mt)){
            mt[to]=v; return true; }
    }
    return false;
}
int matching(vector<vector<int>>& adj, int nRight){
    vector<int> mt(nRight,-1); int res=0;
    for(int v=0;v<(int)adj.size();v++){
        vector<int> used(nRight,0);
        if(try_kuhn(v,adj,used,mt)) res++;
    }
    return res;
}
```

```
struct Dinic {
    struct E{ int to; ll cap; };
    vector<E> es; vector<vector<int>> g; vector<int> lvl, it;
    Dinic(int n): g(n), lvl(n), it(n) {}
    void add(int a,int b,ll c){
        g[a].push_back(es.size()); es.push_back({b,c});
        g[b].push_back(es.size()); es.push_back({a,0}); }
    bool bfs(int s,int t){
        fill(lvl.begin(),lvl.end(),-1);
        queue<int> q; lvl[s]=0; q.push(s);
        while(!q.empty()){ int v=q.front(); q.pop();
            for(int id: g[v]) if(es[id].cap>0 && lvl[es[id].to]<0){
                lvl[es[id].to]=lvl[v]+1; q.push(es[id].to); } }
        return lvl[t]>=0;
    }
    ll dfs(int v,int t,ll f){
        if(v==t) return f;
        for(int& i=it[v]; i<(int)g[v].size(); i++){
            int id=g[v][i], to=es[id].to;
            if(es[id].cap>0 && lvl[to]==lvl[v]+1){
                ll d=dfs(to,t,min(f,es[id].cap));
                if(d>0){ es[id].cap-=d; es[id^1].cap+=d; return d; }
            }
        }
        return 0;
    }
    ll maxflow(int s,int t){
        ll fl=0;
        while(bfs(s,t)){ fill(it.begin(),it.end(),0);
            while(ll f=dfs(s,t,LLONG_MAX/4)) fl+=f; }
        return fl;
    }
};
```

Max-flow = min-cut. After maxflow, vertices reachable from s in the residual graph (cap>0) are the source side of the cut.
Matching via flow: source→left cap 1, left→right cap 1, right→sink cap 1.

E Standard DP

```
ll knapsack(vector<int>& wt, vector<ll>& val, int W){
    vector<ll> dp(W+1,0);
    for(size_t i=0;i<wt.size();i++){
        for(int w=W; w>=wt[i]; w--) // DESCENDING: item used once
            dp[w]=max(dp[w], dp[w-wt[i]]+val[i]);
    }
    return dp[W];
}
```

The inner loop runs descending on purpose. Ascending silently turns 0/1 knapsack into *unbounded* knapsack. It compiles, runs, and gives a plausible wrong answer.

```
int lis(vector<int>& a){
    vector<int> t;
    for(int x: a){
        // upper_bound instead => longest NON-DECREASING
        auto it=lower_bound(t.begin(),t.end(),x);
        if(it==t.end()) t.push_back(x); else *it=x;
    }
    return t.size();
}
```

```
ll coin_change(vector<int>& c, int S){
    const ll INF=LLONG_MAX/4;
    vector<ll> dp(S+1,INF); dp[0]=0;
    for(int s=1;s<=S;s++){ for(int x: c)
        if(x<=s && dp[s-x]+1<dp[s]) dp[s]=dp[s-x]+1;
    }
    return dp[S];
}
```

```
int lcs(string& a, string& b){
    vector<vector<int>> dp(a.size()+1, vector<int>(b.size()+1,0));
    for(size_t i=1;i<=a.size();i++) for(size_t j=1;j<=b.size();j++){
        dp[i][j] = (a[i-1]==b[j-1]) ? dp[i-1][j-1]+1
            : max(dp[i-1][j],dp[i][j-1]);
    }
    return dp[a.size()][b.size()];
}
```

F Number theory

```
vector<int> sieve(int n){
    vector<char> is(n+1,1); is[0]=is[1]=0;
    for(int i=2;(ll)i*i<=n;i++) if(is[i])
        for(int j=i*i;j<=n;j+=i) is[j]=0;
    vector<int> pr;
    for(int i=2;i<=n;i++) if(is[i]) pr.push_back(i);
    return pr;
}
```

```
int gauss(vector<vector<double>> a, vector<double>& ans){
    int n=a.size(), m=a[0].size()-1;
    vector<int> where(m,-1);
    for(int col=0,row=0; col<m && row<n; col++){
        int sel=row;
        for(int i=row;i<n;i++){
            if(fabs(a[i][col])>fabs(a[sel][col])) sel=i;
            if(fabs(a[sel][col])<1e-9) continue;
            swap(a[sel],a[row]);
            where[col]=row;
            for(int i=0;i<n;i++) if(i!=row){
                double c=a[i][col]/a[row][col];
                for(int j=col;j<=m;j++) a[i][j]-=a[row][j]*c;
            }
            row++;
        }
        ans.assign(m,0);
        for(int i=0;i<m;i++) if(where[i]!=-1)
            ans[i]=a[where[i]][m]/a[where[i]][i];
        for(int i=0;i<n;i++){
            double sum=0;
            for(int j=0;j<m;j++) sum+=ans[j]*a[i][j];
            if(fabs(sum-a[i][m])>1e-6) return 0; // no solution
        }
        for(int i=0;i<m;i++) if(where[i]==-1) return 2; // infinitely many
        return 1;
    }
}
```

Returns 0 = no solution, 1 = unique, 2 = infinitely many.

G Geometry additions

```
typedef pair<ll,ll> Pt;
ll cross(Pt O, Pt A, Pt B){
    return (A.first-O.first)*(B.second-O.second)
        - (A.second-O.second)*(B.first-O.first);
}
vector<Pt> hull(vector<Pt> p){
    sort(p.begin(),p.end()); p.erase(unique(p.begin(),p.end()),p.end());
    int n=p.size(), k=0;
    if(n<3) return p;
    vector<Pt> h(2*n);
    for(int i=0;i<n;i++){ // lower
        while(k>=2 && cross(h[k-2],h[k-1],p[i])<=0) k--;
        h[k++]=p[i];
    }
    for(int i=n-2,t=k+1;i>=0;i--){ // upper
        while(k==t && cross(h[k-2],h[k-1],p[i])<=0) k--;
        h[k++]=p[i];
    }
    h.resize(k-1);
    return h;
}
```

Counter-clockwise. <=0 drops collinear points, <0 keeps them. All integer arithmetic — no floating point, so no precision bugs.

Euler’s formula

V – E + F = C + 1, C = number of connected components. Use it whenever you need the number of faces of a planar subdivision.

H String matching

```
vector<int> prefix_function(string& s){
    int n=s.size(); vector<int> pi(n,0);
    for(int i=1;i<n;i++){
        int j=pi[i-1];
        while(j>0 && s[i]!=s[j]) j=pi[j-1];
        if(s[i]==s[j]) j++;
        pi[i]=j;
    }
    return pi;
}
// occurrences of pat in txt: run prefix_function on pat + '\1' + txt
vector<int> kmp_find(string pat, string txt){
    string s = pat + '\1' + txt;
    vector<int> pi = prefix_function(s), res;
    for(size_t i=pat.size()+1;i<s.size();i++){
        if(pi[i]==(int)pat.size()) res.push_back(i - 2*pat.size());
    }
    return res;
}
```

```
struct Trie {
    vector<array<int,26>> ch; vector<int> cnt;
    Trie(){ newNode(); }
    int newNode() { ch.push_back({}); ch.back().fill(-1);
        cnt.push_back(0); return ch.size()-1; }
    void insert(const string& s){
        int v=0;
        for(char c: s){ int x=c-'a';
            if(ch[v][x]==-1) ch[v][x]=newNode();
            v=ch[v][x]; }
        cnt[v]++;
    }
    bool contains(const string& s){
        int v=0;
        for(char c: s){
            int x=c-'a';
            if(ch[v][x]==-1) return false;
            v=ch[v][x];
        }
        return cnt[v]>0;
    }
};
```

I Python

Allowed but not recommended. If someone submits Python, these are the differences from the C++ on the rest of this sheet.

These are secretly O(n) — they create O(n²)

You write	Cost	Use instead
lst.pop(0), insert(0,x)	O(n)	collections.deque
x in lst	O(n)	x in set
s += chunk in a loop	O(n²)	''.join(parts)
a[i:j] in a loop	O(j-i) copy	indices
copying state in recursion	O(n) per node	mutate + undo

Traps and free wins

- **Recursion limit is 1000, no TCO.** DFS on 10⁵ nodes is a Runtime Error. The template below survives depth 200 000.
- **Ints are arbitrary precision** — overflow cannot happen. No long long needed.
- **But // and % round toward −∞** where C++ truncates. -7//2 == -4 in Python, -3 in C++. This bites exactly when you transcribe a C++ formula off this sheet.
- **Use math.isqrt(n), never int(n**0.5)** — the float version is wrong at large n, and the “check until √n” primality test walks into it.
- **stdlib:** heapq (Dijkstra), bisect, functools.lru_cache, itertools.permutations/combinations/product, collections.Counter/defaultdict/deque, math.gcd/lcm/isqrt/comb, pow(a,-1,m), pow(a,b,m).

Template

```
import sys
import math
from collections import deque, defaultdict, Counter
from heapq import heappush, heappop, heapify
from bisect import bisect_left, bisect_right, insort
from itertools import permutations, combinations, product, accumulate
from functools import lru_cache

# ----- INPUT
# All of stdin, read once, split in C. Never call input() below.
_data = sys.stdin.buffer.read()
_it = iter(_data.split())

def ni():      return int(next(_it))          # one int
def nf():      return float(next(_it))        # one float
def ns():      return next(_it).decode()      # one token
def nints(k):  return [int(next(_it)) for _ in range(k)]
def nall():    return list(map(int, _it))      # all remaining ints

# LINE MODE - only when the input has strings containing spaces.
# If you uncomment it, do NOT use ni()/ns()/nints() anywhere: the
# two readers walk the same buffer, and mixing them loses input.
# _lines = iter(_data.splitlines())
# def rline(): return next(_lines).decode()

# ----- OUTPUT
# Buffered: one syscall at the end instead of one per line.
_out = []
def emit(*args): _out.append(' '.join(map(str, args)))

# ----- DEBUG
# stderr is ignored by the judge: safe to leave in on submit.
DEBUG = False
def dbg(*args):
    if DEBUG: print(*args, file=sys.stderr, flush=True)

# ----- SOLVE
def solve():
    # All state lives HERE, as locals. The most common WA is stale
    # data carried between test cases; locals make that impossible.

    n = ni()
    a = nints(n)

    emit(sum(a))

def main():
    # Single test case? Delete the next two lines and just call solve().
    t = ni()
    for _ in range(t):
        solve()

# ----- BOOT
def _run():
    sys.setrecursionlimit(1 << 20) # default 1000: deep DFS = RTE
    main()
    if _out:
        sys.stdout.write('\n'.join(_out))
        sys.stdout.write('\n')

if __name__ == '__main__':
    # Python's C stack, not the recursion counter, is the real limit.
    # A thread with a big stack is what makes deep recursion survive.
    import threading
    try:
        threading.stack_size(1 << 27) # 128 MB
        _t = threading.Thread(target=_run)
        _t.start()
        _t.join()
    except (ValueError, RuntimeError): # platform refused it
        _run()
```

J Before you submit

- ☐ **Loop bounds** — especially in DP.
- ☐ **Array bounds** — big enough? n+1?
- ☐ **Re-initialise between test cases.** Clear vectors, reset globals. Only shows on multi-case input.
- ☐ **Output format** — spelling, capitals, newlines. IMPOSSIBLE vs Impossible.
- ☐ **Overflow** — long long. Does an intermediate product exceed 2³¹?
- ☐ **Index offset** — A[i] or A[i-1]?
- ☐ **Read the complete input**, even once the answer is known.
- ☐ **Precision** — use double, setprecision; many multiplications → logs. Avoid sqrt(-1), log(0).
- ☐ **Rounding** — exactly as the statement prescribes.
- ☐ **Boundary cases** — n=1, empty, all-equal, maximum values.

Verdict triage

Verdict	Look at
Runtime error	array index out of bounds, then sqrt(-1), stack overflow
Time limit	algorithm too slow (you mis-estimated) or an infinite loop
Wrong answer	the checklist above; parse the input first, print to stderr

Tactics

One person writes the bug-free template first · skim **all** problems immediately · check the scoreboard to find the easy ones · solve easy→hard · think it through completely before touching the keyboard · whoever is not coding writes hard test cases · pencode when the machine is busy · don't start with long simulation problems.